DYNAMIC PROGRAMMING — ALL 23 PATTERNS

Java • Clues • Easy / Medium / Hard • State • Transition • Code Template

THE DP MASTER MENTAL MODEL

Core idea	DP is not just “recursion + memoization”. The key is finding a small state that captures everything from the past that can affect the future.
Question 1	What are my choices?
Question 2	What information from the past affects my future? → that is the state.
Question 3	Can two different histories reach the same state? → overlapping subproblems.
Question 4	What is the transition from state → next state?
Question 5	What are the base cases?
Objective	Choose min / max / count / boolean / probability.

PATTERNS 1–2

1. 1D / Linear DP

State: dp[i]
Clues: Array/string, move left→right, previous 1–2 states, take/skip.
Easy: Easy: Climbing Stairs (70) • Min Cost Climbing Stairs (746) • House Robber (198)
Medium: Medium: Delete and Earn (740) • Best Time to Buy/Sell Stock (121)
Hard: Hard: Paint House III (1473) • Burst Balloons (312) • Russian Doll Envelopes (354)
Java code template

```
long[] dp = new long[n + 1];
dp[0] = base0; dp[1] = base1;
for (int i = 2; i <= n; i++) {
dp[i] = Math.max(dp[i-1], dp[i-2] + value[i]);
}
return dp[n];
```

2. 2D Grid DP

State: dp[i][j]
Clues: Grid/matrix, paths, min/max sum, moves between cells.
Easy: Easy: Unique Paths (62) • Minimum Path Sum (64)
Medium: Medium: Unique Paths II (63) • Triangle (120)
Hard: Hard: Cherry Pickup II (1463) • Dungeon Game (174)
Java code template

```
long[][] dp = new long[m][n];
dp[0][0] = 1;
for (int i=0; i<m; i++)
for (int j=0; j<n; j++) {
if (i == 0 && j == 0) continue;
dp[i][j] = best(valid(dp, i, j));
}
return dp[m-1][n-1];
```

PATTERNS 3–4

3. 0/1 Knapsack

State: dp[i][capacity]
Clues: Items with weight/value; choose or skip; each item at most once.
Easy: Easy: 0/1 Knapsack (416-style)
Medium: Medium: Partition Equal Subset Sum (416) • Target Sum (494) • Last Stone Weight II (1049)
Hard: Hard: Profitable Schemes (879) • Minimum Cost to Cut a Stick (1547)
Java code template

```
int[][] dp = new int[n+1][cap+1];
for (int i=1; i<=n; i++)
for (int w=0; w<=cap; w++) {
dp[i][w] = dp[i-1][w];
if (wt[i-1] <= w)
dp[i][w] = Math.max(dp[i][w],
dp[i-1][w-wt[i-1]] + val[i-1]);
}
return dp[n][cap];
```

4. Unbounded Knapsack

State: dp[capacity]
Clues: Unlimited copies of an item; coin change / rod cutting style.
Easy: Easy: Coin Change (322)
Medium: Medium: Coin Change II (518) • Combination Sum IV (377)
Hard: Hard: Minimum Cost For Tickets (983) • Rod Cutting
Java code template

```
int[] dp = new int[amount+1];
Arrays.fill(dp, Integer.MAX_VALUE/2);
dp[0] = 0;
for (int a=1; a<=amount; a++)
for (int coin: coins)
if (coin <= a)
dp[a] = Math.min(dp[a], dp[a-coin] + 1);
return dp[amount];
```

PATTERNS 5–6

5. Subsequence DP

State: dp[i] or dp[i][j]
Clues: Subsequence, take/skip, longest/count/existence.
Easy: Easy: Longest Increasing Subsequence (300) • Longest Continuous Increasing Subarray (674)
Medium: Medium: LCS (1143) • Russian Doll Envelopes (354)
Hard: Hard: Maximum Sum Increasing Subsequence • Distinct Subsequences (115)
Java code template

```
int[] dp = new int[n];
Arrays.fill(dp, 1);
for (int i=0; i<n; i++)
for (int j=0; j<i; j++)
if (a[j] < a[i])
dp[i] = Math.max(dp[i], dp[j] + 1);
return Arrays.stream(dp).max().orElse(0);
```

6. String / Prefix DP

State: dp[i]
Clues: Construct/validate prefix s[0..i); cuts, words, decodings.
Easy: Easy: Decode Ways (91)
Medium: Medium: Word Break (139)
Hard: Hard: Word Break II (140) • Palindrome Partitioning II (132)
Java code template

```
boolean[] dp = new boolean[s.length()+1];
dp[0] = true;
for (int i=1; i<=s.length(); i++)
for (int j=0; j<i; j++)
if (dp[j] && dict.contains(s.substring(j,i)))
dp[i] = true;
return dp[s.length()];
```

PATTERNS 7–8

7. Interval DP

State: dp[l][r]
Clues: Range/subarray/substring; split, merge, remove, burst, choose k.
Easy: Easy: Stone Game (877)
Medium: Medium: Palindrome Partitioning II (132)
Hard: Hard: Burst Balloons (312) • Strange Printer (664) • Matrix Chain Multiplication • Merge Stones
Java code template

```
long[][] dp = new long[n][n];
for (int len=2; len<=n; len++)
for (int l=0; l+len-1<n; l++) {
int r=l+len-1;
dp[l][r]=INF;
for (int k=l; k<r; k++)
dp[l][r]=Math.min(dp[l][r],
dp[l][k]+dp[k+1][r]+cost(l,k,r));
}
return dp[0][n-1];
```

8. Interval + Position / Side

State: dp[l][r][side]
Clues: Points on a line; expand interval; current position is left/right endpoint.
Easy: Easy: line-interval movement basics
Medium: Medium: collecting/visiting points on a line
Hard: Hard: elevator / scheduling-by-location variants
Java code template

```
long[][][] dp = new long[n][n][2];
// dp[l][r][0] = interval handled, at left
// dp[l][r][1] = interval handled, at right
// next choice = expand left OR expand right
dp[nL][nR][0] = transitionFromLeft(...);
dp[nL][nR][1] = transitionFromRight(...);
```

PATTERNS 9–10

9. State Machine DP

State: dp[i][state]
Clues: Status/condition changes over time: hold, buy, sell, cooldown, etc.
Easy: Easy: stock one-transaction variants
Medium: Medium: Best Time to Buy/Sell Stock with Cooldown
Hard: Hard: multi-transaction / transaction-fee variants
Java code template

```
long[][] dp = new long[n+1][states];
for (int i=1; i<=n; i++)
for (int st=0; st<states; st++)
dp[i][st] = best(
dp[i-1][st], // stay
transition(dp, i-1, st) // change state
);
return bestFinal(dp[n]);
```

10. Counting DP

State: dp[state]
Clues: Question asks “how many ways?”; sum transitions instead of min/max.
Easy: Easy: Climbing Stairs (70) • Decode Ways (91)
Medium: Medium: Coin Change II (518) • Dice Rolls With Target Sum (1155)
Hard: Hard: Distinct Subsequences (115) • advanced counting state problems
Java code template

```
long[] dp = new long[target+1];
dp[0] = 1;
for (int s=1; s<=target; s++)
for (int choice: choices)
if (choice <= s)
dp[s] += dp[s-choice];
return dp[target];
```

PATTERNS 11–12

11. Probability DP

State: dp[state]
Clues: Values represent probabilities; random transitions.
Easy: Easy: simple random-walk probability
Medium: Medium: Knight Probability in Chessboard (688)
Hard: Hard: New 21 Game (837) • complex stochastic DP
Java code template

```
double[][] dp = new double[m][n];
dp[startR][startC] = 1.0;
for (int step=0; step<K; step++) {
double[][] next = new double[m][n];
for (int r=0; r<m; r++)
for (int c=0; c<n; c++)
for (Move mv: moves)
if (valid(r+mv.dr,c+mv.dc))
next[r+mv.dr][c+mv.dc] +=
dp[r][c] * mv.prob;
dp = next;
}
```

12. DAG / Topological DP

State: dp[node]
Clues: States form a directed acyclic graph; process in topological order.
Easy: Easy: number of paths in DAG
Medium: Medium: longest path / dependency variants
Hard: Hard: weighted longest path in DAG
Java code template

```
long[] dp = new long[n];
dp[start] = 0;
for (int u : topoOrder) {
for (Edge e : graph[u]) {
dp[e.to] = Math.max(dp[e.to],
dp[u] + e.weight);
}
}
return dp[target];
```

PATTERNS 13–14

13. Tree DP

State: dp[node] or dp[node][state]
Clues: Tree/subtree; combine child answers; often postorder.
Easy: Easy: Maximum Depth of Binary Tree (104)
Medium: Medium: House Robber III (337) • Binary Tree Maximum Path Sum (124)
Hard: Hard: Tree Independent Set / constrained node-selection variants
Java code template

```
int[] dfs(TreeNode u) {
if (u == null) return new int[]{0,0};
int[] L = dfs(u.left), R = dfs(u.right);
int take = u.val + L[1] + R[1];
int skip = Math.max(L[0],L[1])
+ Math.max(R[0],R[1]);
return new int[]{skip,take};
}
```

14. Rerooting DP

State: dp[node] + contribution pass
Clues: Find answer for every node as root; reuse parent-side information.
Easy: Easy: tree contribution basics
Medium: Medium: Sum of Distances in Tree (834)
Hard: Hard: all-roots contribution / minimum-height variants
Java code template

```
void dfs1(int u, int p) {
for (int v: g[u]) if (v != p) {
dfs1(v,u);
sub[u] += sub[v];
ans[u] += ans[v] + sub[v];
}
}
void dfs2(int u, int p) {
for (int v: g[u]) if (v != p) {
ans[v] = ans[u] + n - 2L * sub[v];
dfs2(v,u);
}
}
```

PATTERNS 15–16

15. Bitmask DP

State: dp[mask] or dp[mask][i]
Clues: Small N (≈20); subsets/permutations; visit each item once.
Easy: Easy: Counting Bits (338) • subset-state basics
Medium: Medium: Minimum Cost to Visit All Nodes (847)
Hard: Hard: TSP • Assignment • Hamiltonian Path
Java code template

```
int N = 1 << n;
long[][] dp = new long[N][n];
for (long[] row: dp) Arrays.fill(row, INF);
dp[1][0] = 0;
for (int mask=1; mask<N; mask++)
for (int u=0; u<n; u++)
if ((mask&(1<<u))!=0)
for (int v=0; v<n; v++)
if ((mask&(1<<v))==0) {
int next=mask|(1<<v);
dp[next][v]=Math.min(dp[next][v],
dp[mask][u]+cost[u][v]);
}
```

16. Digit DP

State: dp[pos][tight][started][state]
Clues: Count numbers in [L,R] with digit restrictions; N often up to 10^18.
Easy: Easy: Count Numbers with Unique Digits (357)
Medium: Medium: Numbers With Repeated Digits (1012)
Hard: Hard: Count Special Integers • digit-sum / forbidden-pattern variants
Java code template

```
long dfs(int pos, int tight, int started, int state) {
if (pos == digits.length) return started == 1 ? 1 : 0;
if (memo[pos][tight][started][state] != -1)
return memo[pos][tight][started][state];
int limit = tight == 1 ? digits[pos] : 9;
long ans = 0;
for (int d=0; d<=limit; d++)
ans += dfs(pos+1,
tight==1 && d==limit ? 1 : 0,
started==1 || d!=0 ? 1 : 0,
nextState(state,d));
return memo[pos][tight][started][state]=ans;
}
```

PATTERNS 17–18

17. String + Automaton DP

State: dp[position][automatonState]
Clues: Build a string while maintaining pattern-matching state.
Easy: Easy: small forbidden-pattern state machines
Medium: Medium: string construction with finite states
Hard: Hard: forbidden substrings / regular-language constraints / Aho-Corasick + DP
Java code template

```
long[][] dp = new long[n+1][states];
dp[0][startState] = 1;
for (int i=0; i<n; i++)
for (int st=0; st<states; st++)
for (char ch: alphabet) {
int ns = automaton.next(st,ch);
if (!automaton.forbidden(ns))
dp[i+1][ns] += dp[i][st];
}
```

18. Game DP

State: dp[state]
Clues: Two players make optimal choices; maximize score difference or win/lose.
Easy: Easy: simple take-away games
Medium: Medium: Predict the Winner (486) • Stone Game variants
Hard: Hard: Stone Game II / Can I Win (464) • complex game states
Java code template

```
long solve(int l, int r) {
if (l == r) return a[l];
long takeL = a[l] - solve(l+1,r);
long takeR = a[r] - solve(l,r-1);
return Math.max(takeL,takeR);
}
// dp[state] = best score difference
```

PATTERNS 19–20

19. Minimax DP

State: dp[state]
Clues: Current player maximizes; opponent minimizes. Often use -dp[next].
Easy: Easy: simple two-player optimal-choice games
Medium: Medium: Predict the Winner style
Hard: Hard: multi-state adversarial games
Java code template

```
long minimax(State s) {
if (base(s)) return baseValue;
long best = Long.MIN_VALUE;
for (State next: moves(s))
best = Math.max(best, -minimax(next));
return best;
}
```

20. Partition DP

State: dp[i][k]
Clues: Divide array/string into k groups; choose previous cut j.
Easy: Easy: partition into simple segments
Medium: Medium: Partition Array for Maximum Sum (1043)
Hard: Hard: Split Array Largest Sum (410) • Minimum Difficulty of Job Schedule (1335)
Java code template

```
long[][] dp = new long[n+1][k+1];
fill(dp, INF);
dp[0][0] = 0;
for (int groups=1; groups<=k; groups++)
for (int i=1; i<=n; i++)
for (int j=0; j<i; j++)
dp[i][groups] = Math.min(dp[i][groups],
dp[j][groups-1] + cost(j,i));
return dp[n][k];
```

PATTERNS 21–22

21. DP + Binary Search

State: binary search answer + DP feasibility
Clues: Binary-search a candidate answer; DP checks whether it is feasible.
Easy: Easy: monotone feasibility + simple DP
Medium: Medium: Split Array Largest Sum (410)-style hybrids
Hard: Hard: optimization problems where feasibility itself needs DP
Java code template

```
long lo = low, hi = high;
while (lo < hi) {
long mid = lo + (hi-lo)/2;
if (feasible(mid)) hi = mid;
else lo = mid + 1;
}
return lo;
// feasible(x) may itself use DP
```

22. Monotonic Queue / Optimization DP

State: dp[i] = min/max over j + cost(j,i)
Clues: Normal transition is $O(n^2)$; optimize with deque/heap/CHT/segment tree.
Easy: Easy: sliding-window DP
Medium: Medium: deque-optimized transitions
Hard: Hard: convex hull trick / segment-tree optimized DP
Java code template

```
long[] dp = new long[n];
Deque<Integer> dq = new ArrayDeque<>();
for (int i=0; i<n; i++) {
while (!dq.isEmpty() && dq.peekFirst() < i-window)
dq.pollFirst();
dp[i] = value[i] + (dq.isEmpty() ? 0 : dp[dq.peekFirst()]);
while (!dq.isEmpty() && dp[dq.peekLast()] >= dp[i])
dq.pollLast();
dq.addLast(i);
}
```

PATTERNS 23–23

23. State / Bit / Digit Compression DP

State: dp[position][compressedState]
Clues: Compress a large history into a small representation: mask, state, node+mask, position+state.
Easy: Easy: small state-compression examples
Medium: Medium: position + mask / state combinations
Hard: Hard: combined digit+mask, node+mask, automaton+digit DP
Java code template

```
// General compressed-state form
Map<State, Long> memo = new HashMap<>();

long solve(State s) {
if (base(s)) return baseValue(s);
if (memo.containsKey(s)) return memo.get(s);

long ans = initialValue;
for (Choice c : choices(s)) {
ans = combine(ans, solve(next(s,c)));
}
memo.put(s, ans);
return ans;
}
```

DP CHOICE TREE — START → END

START	Read the problem and identify what is being optimized / counted / decided.
1	Are there meaningful choices at each step?
NO	Usually not DP → consider Greedy / Math / Graph / Simulation / other.
YES	Can I break it into smaller subproblems?
NO	Probably not a standard DP.
YES	Do different histories reach the same subproblem/state?
NO	Could be recursion / greedy / divide-and-conquer / other.
YES	Define the smallest state that captures the relevant past.
LINEAR	One index / previous positions → 1D DP.
GRID	Two indices / matrix / paths → 2D Grid DP.
TAKE/SKIP + CAPACITY	Items + weight/value → 0/1 or Unbounded Knapsack.
SUBSET/PARTITION	Subset sum / equal partition → Subset/Knapsack DP.
TWO SEQUENCES	Two strings/sequences → LCS / Subsequence DP.
INCREASING	Increasing/decreasing subsequence → LIS DP.
RANGE [L,R]	Split/merge/remove interval → Interval DP.
POINTS ON LINE	Expand interval + current side → Interval + Position DP.
STATUS	Buy/sell/hold/cooldown/finite condition → State Machine DP.
DAG	Acyclic dependencies → DAG/Topological DP.
TREE	Subtree + combine children → Tree DP.
EVERY NODE AS ROOT	Need answer for every root → Rerooting DP.
SMALL N ≤ ~20	Subset/permutation/visit all → Bitmask DP.
DIGITS / [L,R]	Digit restrictions / huge N → Digit DP.
STRING PATTERN	Automaton/pattern state → String + Automaton DP.
TWO PLAYERS	Optimal adversarial choices → Game / Minimax DP.
K GROUPS	Split into exactly/at most k groups → Partition DP.
MONOTONE ANSWER	Binary-search candidate + feasibility DP → DP + Binary Search.
FAST TRANSITION NEEDED	$O(n^2)$ transition too slow → optimization DP / deque / heap / CHT / segment tree.
COMPRESSED HISTORY	Mask / small state captures history → State Compression DP.
FINAL	State → Choices → Transition → Base Case → Memoize/Tabulate → Optimize → Answer.

23 PATTERNS — LEARNING ORDER

LEVEL 1 — FOUNDATION	1D Linear • 2D Grid • 0/1 Knapsack • Unbounded Knapsack • String/Prefix
LEVEL 2 — CORE INTERVIEW DP	Subsequence • Interval • Interval+Side • State Machine • Counting • Tree
LEVEL 3 — ADVANCED	DAG/Topological • Rerooting • Bitmask • Digit • Game • Minimax • Partition
LEVEL 4 — DP + OPTIMIZATION	DP + Binary Search • Monotonic Queue / Optimization • Automaton • State Compression

THE MOST IMPORTANT CHEAT CODE

Do not ask: “Which DP pattern is this?” first.

Ask: “What are my choices?” → “What past information affects my future?” → that information is my state.

Then choose the pattern whose state shape matches the problem. The same problem can combine multiple patterns.

MIN / MAX / COUNT / BOOLEAN QUICK RULE

Question asks	Combine choices using
Minimum / cheapest / earliest	min(...)
Maximum / longest / most	max(...)
How many ways?	sum(...)
Can it be done?	OR / any choice is true
Must all choices work?	AND / every choice is true
Probability	sum(probability × transition probability)